GUÍA DE LABORATORIO

Comandos Linux para hacking ético con Docker

Sintaxis · Flags habituales · Ejemplos prácticos

Guía de referencia para el laboratorio de hacking ético con Docker y la administración de VPS. Se conservan las 84 secciones del tutorial original, sus explicaciones, sintaxis, flags y ejemplos.

84 secciones · Índice clicable · Acceso alfabético por comando

SSH y redes / Docker y HTTP / Texto y JSON
Usuarios y permisos / Servicios y diagnóstico

Edición de consulta · 9 de noviembre de 2026

Índice por secciones

Pulsa el título o el número de página para ir a una sección. También puedes usar los marcadores del visor PDF. El pie de página permite volver aquí.

1. Cómo leer un comando Linux 5
2. SSH 5
3. Túnel SSH local -L 6
4. Túnel SSH reverso -R 7
5. ss - conexiones y puertos 7
6. Nmap 8
7. Docker 9
8. Docker Compose 10
9. Curl 10
10. curl -i 10
11. curl -I 11
12. curl -s 11
13. curl -o 11
14. curl -w 11
15. Curl POST 11
16. Curl con Authorization 12
17. Métodos HTTP 12
18. Códigos HTTP importantes 12
19. Grep 12
20. grep -i 13
21. grep -n 13
22. grep -r 13
23. grep -v 13
24. grep -o 13
25. grep -E 13
26. grep -oE 14
27. Expresiones regulares - Regex 14
28. Regex útiles 14
29. Encontrar rutas JavaScript 15
30. Buscar rutas API 15
31. sort 15
32. head 16
33. tail 16
34. jq 16
35. jq -r 16
36. Variables Bash 17
37. printf 17
38. Base64 17
39. Netcat nc 18
40. ip 18
41. hostname 19
42. ping 19
43. DNS con dig 19

44. whoami 19
45. id 19
46. Usuarios 20
47. Grupos 20
48. Privilegios con sudo 20
49. Propietarios 21
50. Permisos Linux 21
51. UFW 22
52. Procesos con ps 22
53. pgrep 22
54. kill 23
55. systemctl 23
56. journalctl 23
57. Navegación básica 23
58. Archivos 24
59. find 24
60. which 25
61. whereis 25
62. file 25
63. strings 25
64. wc 25
65. Pipes | 26
66. Redirecciones 26
67. /dev/null 26
68. && 27
69. ; 27
70. || 27
71. tee 27
72. env 27
73. export 27
74. Información del sistema 28
75. Recursos 28
76. Puertos: flujo de diagnóstico recomendado 28
77. Ejemplo completo con Juice Shop 28
78. Ejemplo completo con túnel SSH 29
79. Combinaciones especialmente útiles 29
80. Los flags que más merece la pena memorizar 30
81. Cómo consultar ayuda 31
82. Tabla mental para diagnosticar problemas 32
83. Chuleta rápida 33
84. Lo que conviene dominar primero 34

Índice alfabético de comandos

Cada entrada enlaza a la sección donde se explica el comando. Las variantes de curl y grep tienen además entradas propias en el índice por secciones.

adduser · § 46
apt · § 48
base64 · § 38
cat · § 58
cd · § 57
chmod · § 50
chown · § 49
cp · § 58
curl · § 9
df · § 75
dig · § 43
docker · § 7
docker compose · § 8
docker compose down · § 8
docker compose logs · § 8
docker compose ps · § 8
docker compose up · § 8
docker exec · § 7
docker images · § 7
docker logs · § 7
docker ps · § 7
docker restart · § 7
docker rm · § 7
docker rmi · § 7
docker start · § 7
docker stop · § 7
du · § 75
echo · § 36
env · § 72
export · § 73
file · § 62
find · § 59
free · § 75
grep · § 19
groups · § 47
head · § 32
hostname · § 41
htop · § 75
id · § 45
ip · § 40
journalctl · § 56
jq · § 34
kill · § 54
less · § 58
ls · § 57
man · § 81
mv · § 58
nc / netcat · § 39
nmap · § 6
passwd · § 46
pgrep · § 53
ping · § 42
printf · § 37
ps · § 52
pwd · § 57
regex · § 27
rm · § 58
sort · § 31
ss · § 5
ssh · § 2
ssh -L (local) · § 3
ssh -R (reverso) · § 4
strings · § 63
su · § 46
sudo · § 48
systemctl · § 55
tail · § 33
tee · § 71
top · § 75
touch · § 58
ufw · § 51
uname · § 74
usermod · § 47
wc · § 64
whereis · § 61
which · § 60
whoami · § 44

Esta guía reúne los comandos más utilizados durante un laboratorio de hacking ético con Docker, redes y administración Linux.

Está pensada para trabajar únicamente sobre sistemas propios, laboratorios autorizados y entornos como OWASP Juice Shop, DVWA y contenedores Docker.

1. Cómo leer un comando Linux

La estructura habitual es:

```
comando [opciones] [argumentos]
```

Ejemplo:

```
curl -s http://127.0.0.1:3000
```

Aquí:

```
curl
│
└── comando

-s
│
└── opción o flag

http://127.0.0.1:3000
│
└── argumento
```

Las opciones también se llaman **flags**.

Normalmente comienzan por:

```
-
```

o:

```
--
```

Ejemplo:

```
curl -I
```

y:

```
curl --include
```

2. SSH

SSH permite conectarse de forma segura a otro equipo.

Sintaxis básica:

```
ssh usuario@servidor
```

Ejemplo:

```
ssh jose@192.168.1.50
```

También puedes indicar un puerto diferente:

```
ssh -p 2222 jose@192.168.1.50
```

Flags SSH habituales

Flag	Significado
`-p`	puerto SSH
`-i`	archivo de clave privada
`-L`	túnel local
`-R`	túnel remoto/reverso
`-N`	no ejecutar una shell remota
`-v`	mostrar información de depuración

3. Túnel SSH local -L

Un túnel local permite acceder desde tu ordenador a un servicio que solamente está disponible desde el servidor remoto.

Ejemplo:

```
ssh -L 8080:127.0.0.1:8080 usuario@IP_VPS
```

La estructura es:

```
-L PUERTO_LOCAL:DESTINO:PUERTO_DESTINO
```

En nuestro ejemplo:

```
8080
│
└── puerto que se abrirá en tu PC

127.0.0.1
│
└── destino visto desde el VPS

8080
│
└── puerto del servicio en el VPS
```

Si DVWA está funcionando en:

```
127.0.0.1:8080
```

dentro del VPS, puedes abrir en tu ordenador:

```
http://127.0.0.1:8080
```

El recorrido sería:

```
Tu PC
127.0.0.1:8080
       │
       │ SSH cifrado
       ▼
VPS
127.0.0.1:8080
       │
       ▼
DVWA
```

No necesitas abrir públicamente el puerto 8080 del VPS.

Una forma especialmente útil es:

```
ssh -N -L 8080:127.0.0.1:8080 usuario@IP_VPS
```

`-N` significa:

```
No abrir una terminal remota.
Solo mantener el túnel SSH.
```

Para cerrar el túnel:

```
Ctrl + C
```

4. Túnel SSH reverso -R

El túnel reverso funciona en dirección contraria.

Sintaxis:

```
ssh -R PUERTO_REMOTO:DESTINO:PUERTO_LOCAL usuario@servidor
```

Ejemplo en un laboratorio controlado:

```
ssh -R 9000:127.0.0.1:3000 usuario@servidor
```

Significa:

```
servidor:9000
       │
       │ túnel SSH
       ▼
tu-PC:3000
```

Puede utilizarse legítimamente para exponer temporalmente un servicio de desarrollo a un servidor remoto sin publicar directamente el puerto.

Con `-N`:

```
ssh -N -R 9000:127.0.0.1:3000 usuario@servidor
```

5. ss - conexiones y puertos

`ss` permite ver conexiones de red y puertos abiertos.

Uno de los comandos más útiles:

```
ss -tulpn
```

Flags:

```
-t    conexiones TCP
-u    conexiones UDP
-l    solamente puertos escuchando
-p    proceso que utiliza el puerto
-n    no resolver nombres; mostrar IP y puerto
```

Ejemplo:

```
sudo ss -tulpn
```

Podrías obtener:

```
LISTEN 0 511 0.0.0.0:22
LISTEN 0 511 127.0.0.1:3000
LISTEN 0 511 127.0.0.1:8080
```

Para buscar solamente el puerto `3000`:

```
ss -tulpn | grep 3000
```

6. Nmap

Nmap sirve para descubrir hosts, puertos y servicios.

Uso básico:

```
nmap IP
```

Ejemplo:

```
nmap 127.0.0.1
```

En un laboratorio Docker:

```
nmap 172.18.0.2
```

Flags Nmap más útiles

Flag	Significado
`-p`	seleccionar puerto
`-p-`	todos los puertos TCP
`-sV`	identificar versiones
`-sC`	scripts básicos de Nmap
`-Pn`	asumir que el host está activo
`-O`	intentar identificar sistema operativo
`-v`	salida más detallada
`-oN`	guardar resultado en texto

Buscar un puerto:

```
nmap -p 3000 127.0.0.1
```

Varios puertos:

```
nmap -p 22,80,443,3000 127.0.0.1
```

Rango:

```
nmap -p 1-1000 127.0.0.1
```

Todos:

```
nmap -p- 127.0.0.1
```

Detectar versiones:

```
nmap -sV 127.0.0.1
```

Ejemplo:

```
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH
3000/tcp open  http    Node.js Express
```

Una combinación habitual en laboratorio:

```
nmap -sV -sC -p 3000 127.0.0.1
```

7. Docker

Docker es una parte central de nuestro laboratorio.

Ver contenedores ejecutándose:

```
docker ps
```

Ver también contenedores detenidos:

```
docker ps -a
```

Ejemplo:

```
CONTAINER ID   IMAGE                  PORTS
11275b017e8e   bkimminich/juice-shop 127.0.0.1:3000->3000/tcp
```

Esto:

```
127.0.0.1:3000->3000/tcp
```

significa:

```
VPS:3000
   |
   ▼
contenedor:3000
```

Entrar dentro de un contenedor

```
docker exec -it nombre-contenedor bash
```

Ejemplo:

```
docker exec -it attacker bash
```

Flags:

```
-i    modo interactivo
-t    crea una terminal
```

Juntas:

```
-it
```

Algunos contenedores solamente tienen sh:

```
docker exec -it juice-shop sh
```

Logs

```
docker logs juice-shop
```

Seguir logs en tiempo real:

```
docker logs -f juice-shop
```

`-f`:

```
follow
```

Detener

```
docker stop juice-shop
```

Iniciar:

```
docker start juice-shop
```

Reiniciar:

```
docker restart juice-shop
```

Eliminar:

```
docker rm juice-shop
```

Eliminar imagen:

```
docker rmi imagen
```

Ver imágenes:

```
docker images
```

8. Docker Compose

Cuando tienes varios servicios:

```
docker compose up
```

En segundo plano:

```
docker compose up -d
```

`-d`:

```
detached
```

El terminal queda libre.

Detener el entorno:

```
docker compose down
```

Ver servicios:

```
docker compose ps
```

Logs:

```
docker compose logs
```

Logs continuos:

```
docker compose logs -f
```

9. Curl

`curl` es probablemente uno de los comandos más importantes del curso.

Permite enviar peticiones HTTP directamente desde la terminal.

Petición básica:

```
curl http://127.0.0.1:3000
```

10. curl -i

```
curl -i http://127.0.0.1:3000
```

`-i` incluye las cabeceras HTTP.

Ejemplo:

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1234

<html>
...
```

11. curl -I

Mayúscula:

```
curl -I http://127.0.0.1:3000
```

Solicita principalmente las cabeceras.

Resultado:

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1234
```

Muy útil para comprobar rápidamente un servidor.

12. curl -s

```
curl -s http://127.0.0.1:3000
```

`-s`:

```
silent
```

Elimina información adicional de progreso.

Es especialmente útil cuando combinamos `curl` con otros comandos:

```
curl -s http://127.0.0.1:3000 | grep script
```

13. curl -o

Guardar respuesta:

```
curl -o pagina.html http://127.0.0.1:3000
```

`-o` significa:

```
output
```

Ahora puedes:

```
cat pagina.html
```

14. curl -w

Permite imprimir información adicional.

Ejemplo:

```
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:3000
```

Resultado:

```
200
```

Muy útil para comprobar códigos HTTP.

15. Curl POST

Ejemplo genérico contra una API de laboratorio:

```
curl -X POST \
-H "Content-Type: application/json" \
-d '{"name":"test"}' \
http://127.0.0.1:3000/api/test
```

Aquí:

`-X POST`

selecciona el método HTTP.

`-H`

añade una cabecera.

`-d`

envía datos.

16. Curl con Authorization

Para una API propia o laboratorio:

```
curl -H "Authorization: Bearer TOKEN" \
http://127.0.0.1:3000/api/profile
```

-H añade:

```
Authorization: Bearer TOKEN
```

17. Métodos HTTP

Los principales que encontrarás son:

```
GET
POST
PUT
PATCH
DELETE
```

Ejemplo:

```
curl -X GET http://127.0.0.1:3000/api/products
```

Normalmente GET es el método predeterminado, por lo que basta:

```
curl http://127.0.0.1:3000/api/products
```

18. Códigos HTTP importantes

```
200    OK
201    creado correctamente
204    correcto, sin contenido

301    redirección permanente
302    redirección temporal

400    petición incorrecta
401    no autenticado
403    prohibido
404    recurso no encontrado

500    error interno del servidor
502    Bad Gateway
503    servicio no disponible
```

19. Grep

grep busca texto.

Ejemplo:

```
grep "admin" archivo.txt
```

Buscar dentro de la salida de otro comando:

```
docker ps | grep juice
```

20. grep -i

Ignorar mayúsculas y minúsculas:

```
grep -i "admin" archivo.txt
```

Encontrará:

```
admin
Admin
ADMIN
```

21. grep -n

Mostrar número de línea:

```
grep -n "password" archivo.txt
```

Ejemplo:

```
34:password
89:password
```

22. grep -r

Buscar recursivamente:

```
grep -r "localhost" .
```

`.` significa:

```
directorio actual
```

23. grep -v

Mostrar lo que NO coincide:

```
grep -v "INFO" log.txt
```

Muy útil para eliminar ruido.

24. grep -o

Mostrar únicamente la coincidencia.

Ejemplo:

```
echo 'src="main.js"' | grep -o 'main.js'
```

Resultado:

```
main.js
```

25. grep -E

Activa expresiones regulares extendidas.

```
grep -E 'jpg|png|gif' archivo.txt
```

Equivale también a:

```
egrep
```

aunque hoy es preferible:

```
grep -E
```

26. grep -oE

Una combinación que utilizamos mucho:

```
grep -oE 'href="[^"]+"'
```

-o:

```
mostrar únicamente coincidencia
```

-E:

```
expresión regular extendida
```

Ejemplo:

```
curl -s http://127.0.0.1:3000 |
grep -oE 'src="[^"]+"'
```

Puede producir:

```
src="runtime.js"
src="polyfills.js"
src="main.js"
```

27. Expresiones regulares - Regex

Una regex describe un patrón.

Ejemplo:

```
[0-9]+
```

Significa:

```
[0-9]    cualquier número

+        una o más veces
```

Encontraría:

```
1
123
5000
```

28. Regex útiles

Cualquier carácter:

```
.
```

Cero o más:

```
*
```

Uno o más:

```
+
```

Cero o uno:

```
?
```

Inicio de línea:

```
^
```

Final:

```
$
```

Números:

```
[0-9]
```

Letras:

```
[a-zA-Z]
```

Negación dentro de corchetes:

```
[^"]
```

significa:

```
cualquier carácter excepto "
```

Por eso:

```
"[^"]+"
```

encuentra contenido entre comillas.

29. Encontrar rutas JavaScript

Ejemplo:

```
curl -s http://127.0.0.1:3000 |
grep -oE 'src="[^"]+\.js"'
```

Podría producir:

```
src="runtime.js"
src="polyfills.js"
src="main.js"
```

30. Buscar rutas API

En una aplicación propia de laboratorio:

```
grep -oE '/api/[A-Za-z0-9/_-]+' main.js
```

Después:

```
grep -oE '/api/[A-Za-z0-9/_-]+' main.js | sort -u
```

31. sort

Ordenar:

```
sort archivo.txt
```

Ordenar y eliminar duplicados:

```
sort -u archivo.txt
```

`-u`:

```
unique
```

Ejemplo:

```
printf "banana\nmanzana\nbanana\n" | sort -u
```

Resultado:

```
banana
manzana
```

32. head

Mostrar las primeras líneas:

```
head archivo.txt
```

Por defecto muestra 10.

Mostrar 20:

```
head -n 20 archivo.txt
```

33. tail

Mostrar las últimas líneas:

```
tail archivo.log
```

Seguir un log en tiempo real:

```
tail -f archivo.log
```

Muy útil para:

```
Nginx
Node.js
servicios Linux
logs de seguridad
```

Por ejemplo:

```
sudo tail -f /var/log/nginx/access.log
```

34. jq

`jq` procesa JSON.

Ejemplo:

```
echo '{"name":"Jose","age":30}' | jq
```

Salida formateada:

```
{
  "name": "Jose",
  "age": 30
}
```

Seleccionar un campo:

```
echo '{"name":"Jose"}' | jq '.name'
```

Resultado:

```
"Jose"
```

35. jq -r

```
jq -r '.name'
```

`-r`:

```
raw
```

Elimina las comillas.

Sin `-r`:

```
"Jose"
```

Con `-r`:

```
Jose
```

Muy útil para variables:

```
TOKEN=$(cat respuesta.json | jq -r '.token')
```

Después:

```
echo "$TOKEN"
```

36. Variables Bash

Crear una variable:

```
NOMBRE="Jose"
```

Mostrar:

```
echo "$NOMBRE"
```

Guardar la salida de un comando:

```
IP=$(hostname -I)
```

Mostrar:

```
echo "$IP"
```

37. printf

`printf` imprime texto con mucho más control que `echo`.

Ejemplo:

```
printf "Hola\n"
```

`\n` significa:

```
salto de línea
```

Ejemplo:

```
printf "Usuario: %s\n" "jose"
```

Resultado:

```
Usuario: jose
```

38. Base64

Codificar:

```
printf 'hola' | base64
```

Resultado parecido a:

```
aG9sYQ==
```

Decodificar:

```
printf 'aG9sYQ==' | base64 -d
```

Resultado:

```
hola
```

`-d`:

```
decode
```

Esto resulta útil para comprender formatos como JWT, donde algunas partes usan codificación Base64URL.

Codificación no significa cifrado.

39. Netcat nc

Netcat sirve para comprobar conexiones TCP/UDP y enviar datos entre servicios en un laboratorio.

Comprobar un puerto:

```
nc -vz 127.0.0.1 3000
```

Flags:

```
-v    verbose
-z    comprobar puerto sin enviar datos
```

Resultado:

```
Connection to 127.0.0.1 3000 port [tcp/*] succeeded!
```

Muy útil para comprobar:

```
nc -vz servidor 22
nc -vz servidor 80
nc -vz servidor 443
```

Escuchar en un puerto dentro de un entorno controlado:

```
nc -l 9000
```

`-l`:

```
listen
```

En otra terminal:

```
nc 127.0.0.1 9000
```

Ahora puedes escribir texto entre las dos terminales.

40. ip

Mostrar interfaces:

```
ip addr
```

Abreviado:

```
ip a
```

Mostrar rutas:

```
ip route
```

o:

```
ip r
```

Ejemplo:

```
default via 192.168.1.1 dev eth0
```

41. hostname

Nombre del equipo:

```
hostname
```

Mostrar direcciones IP:

```
hostname -I
```

42. ping

Comprobar conectividad:

```
ping 8.8.8.8
```

Enviar solamente cuatro paquetes:

```
ping -c 4 8.8.8.8
```

`-c`:

`count`

43. DNS con dig

Consultar DNS:

```
dig example.com
```

Mostrar solamente la respuesta:

```
dig +short example.com
```

Ejemplo:

```
93.184.216.34
```

Consultar un registro específico:

```
dig A example.com
```

```
dig MX example.com
```

```
dig TXT example.com
```

44. whoami

Muestra el usuario actual:

```
whoami
```

Ejemplo:

```
jose
```

45. id

Información del usuario:

```
id
```

Resultado:

```
uid=1000(jose) gid=1000(jose) groups=1000(jose),27(sudo)
```

Esto permite ver:

```
UID
grupo principal
grupos adicionales
```

46. Usuarios

Crear usuario:

```
sudo adduser laboratorio
```

Cambiar contraseña:

```
sudo passwd laboratorio
```

Cambiar de usuario:

```
su - laboratorio
```

47. Grupos

Ver grupos:

```
groups
```

Ver grupos de otro usuario:

```
groups jose
```

Añadir usuario a un grupo:

```
sudo usermod -aG docker jose
```

Aquí:

```
-a    append, añadir sin eliminar otros grupos
```

```
-G    grupos suplementarios
```

Muy importante:

No utilices solamente:

```
usermod -G docker jose
```

porque podrías sustituir sus grupos actuales.

48. Privilegios con sudo

Ejecutar como administrador:

```
sudo comando
```

Ejemplo:

```
sudo apt update
```

Abrir shell root:

```
sudo -i
```

Ver qué puede ejecutar el usuario con sudo:

```
sudo -l
```

49. Propietarios

Ver:

```
ls -l
```

Ejemplo:

```
-rw-r--r-- 1 jose jose 2048 archivo.txt
```

Cambiar propietario:

```
sudo chown jose archivo.txt
```

Propietario y grupo:

```
sudo chown jose:jose archivo.txt
```

Recursivo:

```
sudo chown -R jose:jose directorio/
```

-R:

```
recursive
```

50. Permisos Linux

Ejemplo:

```
-rwxr-xr--
```

Se divide:

```
rwx    propietario
r-x    grupo
r--    otros
```

Donde:

```
r    read
w    write
x    execute
```

Valores:

```
r = 4
w = 2
x = 1
```

Por ejemplo:

```
chmod 755 script.sh
```

significa:

```
7 = rwx
5 = r-x
5 = r-x
```

Para un archivo privado:

```
chmod 600 clave
```

Resultado:

```
rw-------
```

51. UFW

UFW administra el firewall de Ubuntu.

Estado:

```
sudo ufw status
```

Más información:

```
sudo ufw status verbose
```

Antes de activar UFW en un VPS asegúrate de permitir SSH:

```
sudo ufw allow OpenSSH
```

Después:

```
sudo ufw enable
```

Permitir HTTP:

```
sudo ufw allow 80/tcp
```

HTTPS:

```
sudo ufw allow 443/tcp
```

Eliminar regla:

```
sudo ufw delete allow 3000/tcp
```

En un laboratorio con Docker es perfectamente razonable dejar aplicaciones como Juice Shop ligadas solamente a:

```
127.0.0.1
```

y acceder a ellas mediante túneles SSH.

Eso evita exponerlas directamente a Internet.

52. Procesos con ps

Mostrar procesos:

```
ps aux
```

Buscar Node:

```
ps aux | grep node
```

Buscar Docker:

```
ps aux | grep docker
```

53. pgrep

Más limpio que:

```
ps aux | grep nginx
```

puedes utilizar:

```
pgrep nginx
```

Con información:

```
pgrep -a nginx
```

54. kill

Finalizar proceso:

```
kill PID
```

Ejemplo:

```
kill 1234
```

Si no responde:

```
kill -9 1234
```

`-9` envía `SIGKILL`.

Conviene utilizarlo solamente cuando una terminación normal no funciona.

55. systemctl

Gestionar servicios Linux.

Estado:

```
sudo systemctl status nginx
```

Iniciar:

```
sudo systemctl start nginx
```

Detener:

```
sudo systemctl stop nginx
```

Reiniciar:

```
sudo systemctl restart nginx
```

Recargar configuración:

```
sudo systemctl reload nginx
```

Activar al arrancar:

```
sudo systemctl enable nginx
```

56. journalctl

Consultar logs de systemd.

Por ejemplo:

```
sudo journalctl -u nginx
```

Últimos mensajes:

```
sudo journalctl -u nginx -n 50
```

Seguir en tiempo real:

```
sudo journalctl -u nginx -f
```

57. Navegación básica

Directorio actual:

```
pwd
```

Listar:

```
ls
```

Detalles:

```
ls -l
```

Incluyendo archivos ocultos:

```
ls -la
```

Entrar:

```
cd directorio
```

Subir un nivel:

```
cd ..
```

Ir a tu home:

```
cd ~
```

58. Archivos

Crear:

```
touch archivo.txt
```

Ver:

```
cat archivo.txt
```

Ver paginado:

```
less archivo.txt
```

Copiar:

```
cp origen destino
```

Mover:

```
mv origen destino
```

Eliminar:

```
rm archivo
```

Eliminar directorio:

```
rm -r directorio
```

Ten especial cuidado con:

```
rm -rf
```

`-r` significa recursivo.

`-f` significa forzar.

Un comando incorrecto ejecutado como root puede eliminar una gran cantidad de datos.

59. find

Buscar archivos.

Ejemplo:

```
find /var/www -name "*.js"
```

Buscar solamente archivos:

```
find . -type f
```

Directorios:

```
find . -type d
```

Buscar archivos `.log`:

```
find /var/log -type f -name "*.log"
```

60. which

Ver dónde está instalado un comando:

```
which curl
```

Ejemplo:

```
/usr/bin/curl
```

61. whereis

Más información:

```
whereis nginx
```

Puede mostrar:

```
nginx: /usr/sbin/nginx /etc/nginx /usr/share/nginx
```

62. file

Identificar un archivo:

```
file archivo
```

Ejemplo:

```
file programa
```

Salida posible:

```
ELF 64-bit LSB executable
```

63. strings

Extraer cadenas legibles de un archivo:

```
strings archivo
```

Buscar después:

```
strings archivo | grep -i http
```

Es útil para análisis básico de binarios propios o archivos de laboratorio.

64. wc

Contar líneas:

```
wc -l archivo.txt
```

Palabras:

```
wc -w archivo.txt
```

Bytes:

```
wc -c archivo.txt
```

También:

```
grep ERROR log.txt | wc -l
```

para contar errores.

65. Pipes |

El pipe es una de las ideas más importantes de Linux.

```
comando1 | comando2
```

La salida del primero pasa al segundo.

Ejemplo:

```
docker ps | grep juice
```

Otro:

```
curl -s http://127.0.0.1:3000 |
grep -oE 'src="[^"]+\.js"' |
sort -u
```

El recorrido es:

```
curl
  ↓
HTML
  ↓
grep
  ↓
archivos JavaScript
  ↓
sort -u
  ↓
resultado limpio
```

66. Redirecciones

Guardar salida:

```
comando > archivo.txt
```

Esto sobrescribe el archivo.

Añadir:

```
comando >> archivo.txt
```

Ejemplo:

```
nmap -sV 127.0.0.1 > scan.txt
```

67. /dev/null

Es como un agujero negro.

Todo lo enviado allí desaparece.

Ejemplo:

```
curl -s -o /dev/null http://127.0.0.1:3000
```

Muy útil combinado con:

```
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:3000
```

68. &&

Ejecuta el siguiente comando solamente si el anterior funcionó.

```
sudo apt update && sudo apt upgrade
```

Si `apt update` falla, `apt upgrade` no se ejecutará.

69. ;

Ejecuta el siguiente comando independientemente del resultado anterior:

```
comando1 ; comando2
```

70. ||

Ejecuta el segundo comando si el primero falla.

Ejemplo:

```
curl http://127.0.0.1:3000 || echo "Servidor no disponible"
```

71. tee

Permite mostrar y guardar a la vez.

```
comando | tee resultado.txt
```

Ejemplo:

```
nmap -sV 127.0.0.1 | tee scan.txt
```

Verás el resultado en pantalla y además quedará guardado.

Añadir en lugar de sobrescribir:

```
tee -a archivo
```

72. env

Mostrar variables de entorno:

```
env
```

Buscar una concreta:

```
env | grep PATH
```

Mostrar:

```
echo "$PATH"
```

73. export

Crear una variable de entorno:

```
export API_URL="http://127.0.0.1:3000"
```

Después:

```
curl "$API_URL"
```

74. Información del sistema

Kernel:

```
uname -a
```

Distribución:

```
cat /etc/os-release
```

Arquitectura:

```
uname -m
```

75. Recursos

Memoria:

```
free -h
```

`-h`:

```
human readable
```

Disco:

```
df -h
```

Uso de un directorio:

```
du -sh directorio
```

Procesos:

```
top
```

Una alternativa cómoda si está instalada:

```
htop
```

76. Puertos: flujo de diagnóstico recomendado

Cuando una aplicación no responde, una secuencia muy útil es:

```
docker ps
```

Después:

```
ss -tulpn
```

Después:

```
curl -I http://127.0.0.1:3000
```

Después:

```
nc -vz 127.0.0.1 3000
```

Y finalmente, en tu propio laboratorio:

```
nmap -sV -p 3000 127.0.0.1
```

Cada comando comprueba una capa diferente.

77. Ejemplo completo con Juice Shop

Comprobar Docker:

```
docker ps
```

Buscar:

```
docker ps | grep juice
```

Comprobar puerto:

```
ss -tulpn | grep 3000
```

Comprobar HTTP:

```
curl -I http://127.0.0.1:3000
```

Comprobar estado HTTP:

```
curl -s -o /dev/null \
-w "%{http_code}\n" \
http://127.0.0.1:3000
```

Descargar página:

```
curl -s http://127.0.0.1:3000 > pagina.html
```

Buscar JavaScript:

```
grep -oE 'src="[^"]+\.js"' pagina.html
```

Eliminar duplicados:

```
grep -oE 'src="[^"]+\.js"' pagina.html | sort -u
```

78. Ejemplo completo con túnel SSH

Imaginemos que Juice Shop está:

```
VPS
127.0.0.1:3000
```

Por seguridad no exponemos el `3000` públicamente.

Desde nuestro PC:

```
ssh -N -L 3000:127.0.0.1:3000 usuario@IP_VPS
```

Después abrimos:

```
http://127.0.0.1:3000
```

La aplicación viaja mediante el túnel SSH.

No necesitas una regla:

```
ufw allow 3000
```

De hecho, mantenerlo ligado a `127.0.0.1` es preferible para un laboratorio vulnerable.

79. Combinaciones especialmente útiles

Encontrar un proceso:

```
ps aux | grep nginx
```

Puertos abiertos:

```
sudo ss -tulpn
```

Buscar puerto concreto:

```
sudo ss -tulpn | grep 3000
```

Comprobar servicio HTTP:

```
curl -I http://127.0.0.1:3000
```

Código HTTP:

```
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:3000
```

JSON bonito:

```
curl -s http://127.0.0.1:3000/api/products | jq
```

Extraer propiedad:

```
curl -s URL | jq -r '.name'
```

Buscar texto:

```
grep -i "error" archivo.log
```

Buscar recursivamente:

```
grep -ri "localhost" .
```

Archivos JavaScript:

```
find . -type f -name "*.js"
```

Logs Docker:

```
docker logs -f juice-shop
```

Logs servicio:

```
journalctl -u nginx -f
```

Firewall:

```
sudo ufw status verbose
```

80. Los flags que más merece la pena memorizar

SSH

```
-p     puerto
-i     identidad/clave privada
-L     túnel local
-R     túnel remoto
-N     solamente túnel
-v     verbose
```

Curl

```
-s     silent
-i     incluir cabeceras
-I     solamente cabeceras
-X     método HTTP
-H     cabecera
-d     datos
-o     archivo de salida
-w     formato de salida
-L     seguir redirecciones
```

Grep

```
-i     ignorar mayúsculas
-n     número de línea
-r     recursivo
-v     excluir coincidencia
-o     solamente coincidencia
-E     regex extendida
```

Nmap

```
-p     puerto
-p-    todos los puertos
-sV    detectar versión
-sC    scripts básicos
-Pn    no hacer descubrimiento previo
-O     intentar identificar SO
-v     verbose
-oN    guardar salida
```

Docker

```
-a     incluir detenidos
-it    terminal interactiva
-f     seguir logs
-d     segundo plano en Docker Compose
```

Netcat

```
-l     escuchar
-v     verbose
-z     comprobar puerto
```

Linux general

```
-h     formato legible
-r/-R  recursivo
-f     force/follow según comando
-a     all
-n     número/no resolver nombres según comando
-v     verbose
```

Recuerda que un mismo flag puede significar algo diferente dependiendo del comando.

Por ejemplo:

```
curl -s
```

significa:

```
silent
```

pero:

```
grep -s
```

tiene otro significado.

Por eso siempre conviene pensar:

```
comando + flag
```

como una unidad.

81. Cómo consultar ayuda

La forma rápida:

```
comando --help
```

Ejemplo:

```
curl --help
```

Manual completo:

```
man comando
```

Ejemplo:

```
man ssh
```

Buscar dentro del manual:

```
/
```

Escribes después una palabra:

```
/LocalForward
```

Siguiente resultado:

```
n
```

Salir:

```
q
```

82. Tabla mental para diagnosticar problemas

Cuando algo falla, piensa aproximadamente así:

```
¿Está funcionando el proceso?
        │
        ├── ps
        ├── systemctl
        └── docker ps

        ↓

¿Está escuchando el puerto?
        │
        └── ss -tulpn

        ↓

¿Puedo conectar?
        │
        └── nc -vz

        ↓

¿Responde HTTP?
        │
        └── curl

        ↓

¿Qué servicio/version hay?
        │
        └── nmap -sV

        ↓

¿Qué dicen los logs?
        │
        ├── docker logs
        ├── journalctl
        └── tail
```

Esta forma de pensar es mucho más importante que memorizar cientos de comandos.

83. Chuleta rápida

```
# Usuario
whoami
id
groups

# Sistema
uname -a
cat /etc/os-release

# Red
ip a
ip r
ss -tulpn
ping -c 4 IP

# DNS
dig +short dominio

# Puertos
nc -vz IP PUERTO
nmap -sV -p PUERTO IP

# HTTP
curl URL
curl -I URL
curl -s URL
curl -s URL | jq

# Archivos
pwd
ls -la
cat archivo
less archivo
find . -name "*.js"

# Texto
grep -i palabra archivo
grep -ri palabra .
grep -oE 'regex' archivo
sort -u
head
tail

# Docker
docker ps
docker ps -a
docker logs -f contenedor
docker exec -it contenedor bash
docker stop contenedor
docker start contenedor

# Docker Compose
docker compose up -d
docker compose ps
docker compose logs -f
docker compose down

# Servicios
systemctl status nginx
journalctl -u nginx -f

# Firewall
sudo ufw status verbose

# SSH
```

```
ssh usuario@IP
ssh -p PUERTO usuario@IP

# Túnel local
ssh -N -L 3000:127.0.0.1:3000 usuario@IP_VPS

# Túnel remoto
ssh -N -R 9000:127.0.0.1:3000 usuario@SERVIDOR

# Recursos
free -h
df -h
du -sh directorio
top
```

84. Lo que conviene dominar primero

Para el curso de hacking ético con Docker, los comandos prioritarios son:

```
docker
curl
grep
jq
nmap
ss
ssh
nc
ip
find
cat
less
head
tail
sort
printf
chmod
chown
sudo
ufw
systemctl
journalctl
```

Pero todavía más importante es acostumbrarte a combinarlos:

```
curl -s URL | grep PATRON

curl -s URL | jq

grep PATRON archivo | sort -u

docker ps | grep nombre

ss -tulpn | grep puerto

journalctl -u servicio -f
```

En Linux, gran parte del poder está precisamente en combinar comandos pequeños mediante pipes.

Esta guía puede servir como base del curso. Según se avance con Juice Shop, DVWA, Docker, redes, SSH y Nginx, se pueden añadir debajo de cada comando los ejemplos reales del laboratorio.